ICD-10 Code Prediction from MIMIC-IV Discharge Summaries

Rohan Kumar Chitra Deva Sai Sunder Tangella

NLP Final Project — Spring 2025

April 23, 2026

Individual Contributions

Rohan Kumar Chitra — *Approach 1: Chunk-BERT + Label Attention*

- Designed sliding-window chunking strategy (6×512 tokens, stride 256) for full-document coverage.
- Implemented per-label attention with 50 learnable query vectors over Bio_ClinicalBERT.
- Two-phase training schedule (frozen encoder → fine-tune last 2 BERT layers).
- MIMIC-IV data extraction pipeline, cohort construction (122K admissions).
- Text preprocessing: de-identification stripping, normalization, TF-IDF feature engineering.
- Built shared evaluation framework (threshold tuning, head/torso/tail analysis).

Deva Sai Sunder Tangella — *Approach 2: BiLSTM-LAAT*

- Implemented LAAT architecture: BiLSTM encoder + structured label attention mechanism.
- Word-level tokenization with 50K vocabulary, processing up to 4,000 tokens natively.
- Temperature calibration post-training (ECE reduced from 0.07 to 0.02).
- Comparative evaluation across all models (scatter analysis, bar charts).
- Production packaging: `src/` Python module, FastAPI API, Docker containerization.
- Pytest test suite covering data processing, model shapes, and API schemas.

Shared: Baseline Model A (TF-IDF + SGD), ensemble experiments, Streamlit demo, report.

Abstract

Automated prediction of ICD-10 diagnosis codes from clinical discharge summaries is a critical task for reducing the $25B annual cost of manual medical coding in the United States. We address this as a multi-label classification problem using the MIMIC-IV clinical database, predicting the top-50 most frequent ICD-10 codes from 122,304 hospital admissions. We establish a strong TF-IDF + SGD baseline (Micro-F1 = 0.60) and propose two neural approaches: (1) a Chunk-BERT architecture with per-label attention that processes full documents via overlapping 512-token windows over Bio_ClinicalBERT, and (2) a BiLSTM with Label-Aware Attention (LAAT) that encodes up to 4,000 word tokens in a single pass. The BiLSTM-LAAT model achieves the best individual Micro-F1 of 0.72, a +0.12 improvement over the baseline, with balanced precision (0.72) and recall (0.72). A weighted ensemble of TF-IDF and BiLSTM-LAAT further improves performance to Micro-F1 = 0.72 and Micro-AUROC = 0.95. We provide a complete system including a Streamlit web demo, FastAPI service, and Docker deployment.

1 Introduction

1.1 Motivation

The International Classification of Diseases, 10th Revision (ICD-10), is the global standard for encoding diagnoses and procedures in clinical records. In the United States, every hospital encounter is assigned a set of ICD-10 codes used for billing, epidemiological surveillance, and quality-of-care measurement. This coding process is predominantly manual, performed by trained clinical coders who review lengthy discharge summaries—a labor-intensive process costing approximately $25 billion per year [5]. Automating ICD code prediction from clinical text has the potential to reduce costs, decrease coding errors, and accelerate reimbursement cycles.

1.2 Problem Definition

Given a clinical discharge summary $\mathbf{x}$ (typically 800–2,000 words), the task is to predict the set of ICD-10 diagnosis codes $\mathbf{y} = \{y_1, y_2, \ldots, y_k\}$ assigned to the corresponding hospital encounter. This is formulated as a *multi-label classification* problem, where the label space consists of the top-50 most frequent ICD-10 codes in the MIMIC-IV dataset.

1.3 Challenges

- **Massive label space:** 7,940 unique ICD-10 codes remain after frequency filtering ($\geq$10 occurrences), exhibiting extreme class imbalance.
- **Long documents:** Discharge summaries average $\sim$1,500 words, exceeding BERT's 512-token context window and requiring full-document processing strategies.
- **Semantic complexity:** The same diagnosis can be expressed with diverse clinical phrasing, abbreviations, and negation patterns.
- **Label co-occurrence:** ICD codes exhibit complex co-occurrence patterns (e.g., hypertension frequently co-occurs with diabetes), requiring models to capture inter-label dependencies.

1.4 Contributions

1. A comprehensive multi-label classification pipeline for ICD-10 prediction on MIMIC-IV (Top-50 codes), including data extraction, preprocessing, and patient-level train/val/test splits.
2. Two full-document neural architectures: Chunk-BERT with per-label attention (Approach 1) and BiLSTM-LAAT (Approach 2), both addressing BERT's context-length limitation.
3. Systematic comparison of five individual models and five ensemble configurations, with the BiLSTM-LAAT model achieving a Micro-F1 of 0.72 (+0.12 over the TF-IDF baseline).
4. A production-ready deployment system with a Streamlit web interface, FastAPI backend, and Docker containerization.

2 Prior Work

Automated ICD coding from clinical text has been studied extensively, progressing from rule-based and traditional ML approaches to deep learning architectures.

Traditional approaches. Early work applied bag-of-words representations with SVMs and logistic regression to MIMIC-III discharge notes [6]. TF-IDF features with one-vs-rest classifiers remain competitive baselines due to ICD codes' dependence on specific medical terminology.

CAML and DR-CAML. Mullenbach et al. [5] introduced Convolutional Attention for Multi-Label classification (CAML), using a CNN encoder with per-label attention over clinical text.

The description-regularized variant (DR-CAML) incorporated ICD code descriptions as auxiliary supervision, improving rare-code performance.

PLM-ICD. Huang et al. [2] proposed PLM-ICD, which combines a pretrained language model with overlapping text chunks and per-label attention. By processing the full document through sliding windows and attending to all token positions per label, PLM-ICD achieved state-of-the-art results on MIMIC-III with the full label set. Our Approach 1 is directly inspired by this architecture.

LAAT. Vu et al. [7] proposed Label Attention (LAAT), a BiLSTM encoder with a structured attention mechanism that learns label-specific representations. LAAT demonstrated that RNN-based models with label attention can rival transformer approaches while being significantly more efficient. Our Approach 2 implements this architecture.

ClinicalBERT. Alsentzer et al. [1] released Bio_ClinicalBERT, a BERT model pretrained on MIMIC-III clinical notes. While it captures rich contextual semantics, its 512-token limit discards 40–60% of typical discharge summaries, motivating chunking strategies.

Focal Loss. Lin et al. [4] introduced focal loss to address class imbalance in object detection. We adopt this loss function for both Approach 1 (v2) and Approach 2 to down-weight well-classified examples and focus learning on rare codes.

3 Methods

We develop and compare five models for multi-label ICD-10 prediction: a TF-IDF baseline (Model A), a truncation-based ClinicalBERT classifier (Model B), a chunk-based ClinicalBERT with label attention (Model C), a BiLSTM with label attention (Model D), and weighted probability ensembles.

3.1 Model A: TF-IDF + SGD (Baseline)

The baseline represents each discharge summary as a TF-IDF vector and trains a linear classifier.

Feature extraction. We apply TF-IDF vectorization with 50,000 features, unigram+bigram range, and sublinear term-frequency scaling $(1 + \log(\text{tf}))$. This produces a sparse feature matrix $\mathbf{X} \in \mathbb{R}^{n \times 50000}$.

Classifier. A `OneVsRestClassifier` wrapping `SGDClassifier` with log-loss (logistic regression), L2 regularization ($C = 1.0$, $\alpha = 1/(C \cdot n)$), and balanced class weights to address label imbalance. Training runs for 100 iterations with all CPU cores.

Threshold tuning. We sweep global thresholds on the validation set to maximize Micro-F1, selecting $t^* = 0.53$.

3.2 Model B: ClinicalBERT (Truncation Baseline)

Model B fine-tunes Bio_ClinicalBERT [1] with a simple classification head to establish a transformer baseline.

Architecture. The `[CLS]` token embedding from Bio_ClinicalBERT (768-dim) passes through dropout ($p = 0.1$) and a linear layer projecting to 50 labels. Input is truncated to 512 tokens via a *smart truncation* strategy that prioritizes clinically informative sections (discharge diagnoses, hospital course) using regex-based section extraction.

Training. AdamW optimizer with lr $= 2 \times 10^{-5}$, linear warmup (10% of steps), weight decay $= 0.01$, BCE with logits loss, batch size 32, 3 epochs, mixed precision (fp16).

3.3 Approach 1: Chunk-BERT + Label Attention (Model C)

Inspired by PLM-ICD [2], Model C addresses the 512-token limitation through overlapping chunks and per-label attention.

Chunking. Each discharge summary is tokenized and split into up to $K = 6$ overlapping chunks of 512 tokens with stride 256 (50% overlap). Shorter documents are zero-padded; a chunk-level mask prevents attention to padding.

Encoding. Bio_ClinicalBERT encodes each chunk independently, producing token embeddings $\mathbf{H}_k \in \mathbb{R}^{512\times 768}$ per chunk. All chunk embeddings are concatenated into a single sequence $\mathbf{H} \in \mathbb{R}^{(K\cdot 512)\times 768}$.

Label attention. We define 50 learnable query vectors $\mathbf{Q} \in \mathbb{R}^{50\times 768}$ (Xavier-initialized). For each label l:

$$\alpha_l = \text{softmax}\left(\frac{\mathbf{Q}_l \cdot \mathbf{H}^\top}{\sqrt{768}}\right), \quad \mathbf{c}_l = \alpha_l \cdot \mathbf{H} \tag{1}$$

where $\alpha_l \in \mathbb{R}^{K\cdot 512}$ are the attention weights and $\mathbf{c}_l \in \mathbb{R}^{768}$ is the label-specific context vector. A per-label linear classifier produces the final logit: $\hat{y}_l = \mathbf{w}_l^\top \mathbf{c}_l + b_l$.

Two-phase training.

- *Phase 1 (frozen):* 5 epochs at lr $= 10^{-3}$ with BERT weights frozen—trains only the attention and classifier layers.
- *Phase 2 (fine-tune):* 7 epochs at lr $= 2 \times 10^{-5}$ with the last 2 BERT layers unfrozen.

Effective batch size is 32 (micro-batch 4 $\times$ gradient accumulation 8). Focal loss ($\gamma = 2.0$, $\alpha = 0.25$) is used in v2 to address class imbalance. Early stopping with patience 3 monitors validation Micro-F1.

3.4 Approach 2: BiLSTM-LAAT (Model D)

Inspired by LAAT [7], Model D uses a BiLSTM encoder with structured label attention, processing documents up to 4,000 word tokens in a single pass. Figure 1 illustrates the end-to-end architecture.

Data flow overview. A raw discharge summary is preprocessed (de-identification removal, lowercasing, whitespace normalization), split on whitespace into words, mapped to integer indices via a 50K-word vocabulary, and padded to length 4,000. The embedding layer maps each index to a 200-dimensional vector. The BiLSTM produces a 512-dim contextual representation per token. The LAAT attention mechanism then computes a separate weighted summary of these representations for each of the 50 ICD labels. Finally, 50 independent per-label classifiers produce sigmoid probabilities, and a global threshold ($t = 0.30$) determines the predicted code set.

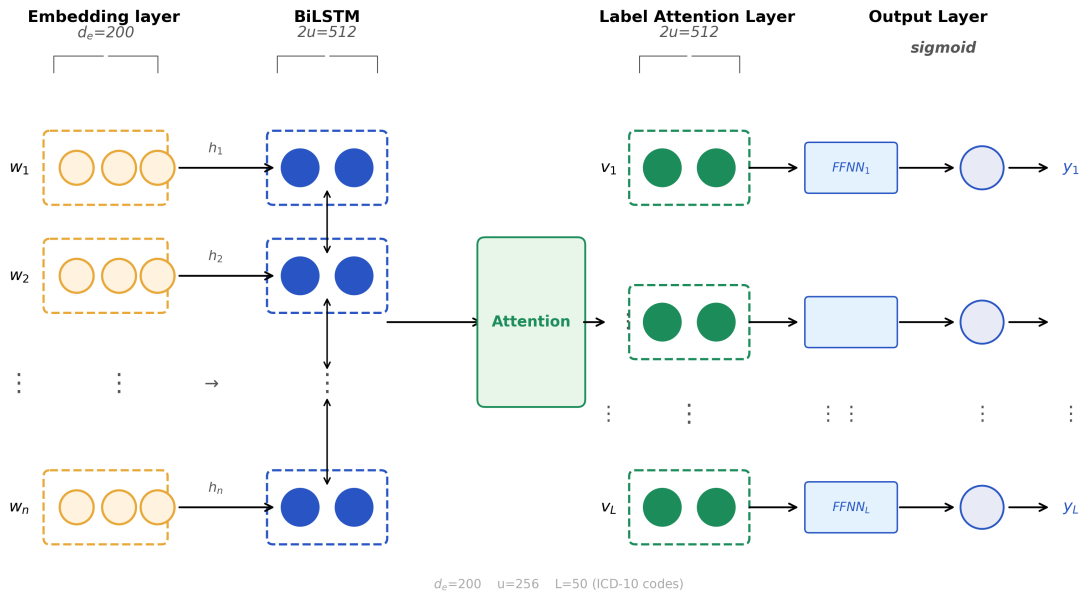

Figure 1: BiLSTM-LAAT architecture. The encoding pipeline (top) maps raw text through tokenization, embedding, and a bidirectional LSTM to produce contextual token representations $\mathbf{H}$. The Label Attention (LAAT) block (middle) computes label-specific document representations: each of the 50 ICD codes attends to different tokens via learned query vectors. Per-label classifiers with sigmoid activation produce the final predictions (bottom). The dashed line shows the skip connection where $\mathbf{H}$ feeds directly into the weighted sum $\mathbf{V} = \mathbf{A} \cdot \mathbf{H}$.

Embedding. A word-level vocabulary of 50,000 tokens is constructed from training data (plus `<PAD>` and `<UNK>`). Each document is tokenized and padded/truncated to 4,000 tokens, then embedded via a trainable embedding layer ($d = 200$).

BiLSTM encoder. A single-layer bidirectional LSTM with hidden size 256 per direction produces contextualized representations $\mathbf{H} \in \mathbb{R}^{T \times 512}$ ($512 = 2 \times 256$).

Label attention (LAAT). Following Vu et al. [7]:

$$\mathbf{Z} = \tanh(\mathbf{W} \cdot \mathbf{H}^\top) \in \mathbb{R}^{d_a \times T}, \quad d_a = 256 \tag{2}$$

$$\mathbf{A} = \text{softmax}(\mathbf{U} \cdot \mathbf{Z}) \in \mathbb{R}^{L \times T} \tag{3}$$

$$\mathbf{V} = \mathbf{A} \cdot \mathbf{H} \in \mathbb{R}^{L \times 512} \tag{4}$$

where $\mathbf{W} \in \mathbb{R}^{d_a \times 512}$ and $\mathbf{U} \in \mathbb{R}^{L \times d_a}$ ($L = 50$ labels) are learnable parameters. Each label's output $\mathbf{v}_l$ is classified independently through a per-label linear layer with sigmoid activation.

Training. AdamW optimizer with lr $= 10^{-3}$ and weight decay $= 0.01$, batch size 32, 10 epochs, focal loss ($\gamma = 2.0$, $\alpha = 0.25$), embedding dropout $p = 0.3$, early stopping patience 3. The learning rate follows a linear warmup for the first 5% of total steps, then linearly decays to zero. Gradient clipping (max_norm $= 1.0$) prevents exploding gradients, a known issue with LSTMs on long sequences. Mixed precision training (torch AMP with `float16` autocast and `GradScaler`) provides $\sim$2$\times$ speedup and halves GPU memory usage. The model converges at epoch 8 with 11.1M trainable parameters (10M in the embedding table alone).

Implementation details. To avoid wasting computation on padding tokens, we use `pack_padded_sequence` before the LSTM and `pad_packed_sequence` after, so the LSTM processes only real tokens. In the label attention layer, padding positions are masked with $-\infty$ before softmax to ensure they receive zero attention weight, preventing the model from attending to meaningless padding.

Temperature calibration. Post-training, we fit a single temperature parameter T on the validation set using the LBFGS optimizer to minimize BCEWithLogitsLoss(logits$/T$, labels). Since $T^* = 0.63 < 1$, dividing logits by T amplifies their magnitude, pushing sigmoid outputs further toward 0 and 1—making the model more decisive. This reduces Expected Calibration Error (ECE) from 0.07 to 0.02, meaning predicted probabilities now accurately reflect true likelihoods.

Parameter count. Table 1 provides a breakdown of Model D's 11.1M trainable parameters. The embedding table dominates (90%), while the LSTM, attention, and classifier layers together comprise only ∼1.1M parameters.

Table 1: Parameter count breakdown for Model D (BiLSTM-LAAT).

Component	**Parameters**	**%**
Embedding (50K × 200)	10,000,000	90.0%
BiLSTM (bidirectional)	935,936	8.4%
Attention **W** (512 × 256)	131,072	1.2%
Attention **U** (256 × 50)	12,800	0.1%
Classifier (512 × 50 + bias)	25,650	0.2%
Total	**11,105,458**	100%

3.5 Ensemble Methods

We explore weighted probability averaging of individual models. Given model predictions $\mathbf{P}_i \in [0,1]^{n \times L}$:

$$\mathbf{P}_{\text{ens}} = \sum_i w_i \cdot \mathbf{P}_i, \quad \sum_i w_i = 1 \tag{5}$$

Weights w_i and the global decision threshold t are jointly tuned on the validation set to maximize Micro-F1. We evaluate five ensemble configurations (Table 2).

Table 2: Ensemble configurations with validation-tuned weights and thresholds.

Variant	**Components**	w_A	w_{C1}	w_{C2}	w_D	**Threshold**
v1	A + C v1	0.65	0.35	—	—	0.63
v2	A + C v2	0.35	—	0.65	—	0.43
v3	A + C v1 + C v2	0.30	0.00	0.70	—	0.40
v4	A + D	0.15	—	—	0.85	0.35
v5	A + C v2 + D	0.10	—	0.30	0.60	0.35

4 Dataset

4.1 Data Source

We use the MIMIC-IV clinical database [3], a de-identified dataset of electronic health records from Beth Israel Deaconess Medical Center. Data is extracted from two primary tables:

- `discharge`: 331,793 discharge summary notes.
- `diagnoses_icd`: 6,364,488 diagnosis code assignments.

After joining on admission identifiers and removing entries without both notes and ICD-10 codes, the final cohort comprises **122,304 admissions**.

4.2 Label Statistics

Table 3: Dataset statistics overview.

Statistic	Value
Total discharge notes	331,793
Admissions with notes + ICD	122,304
Unique ICD-10 codes (freq $\geq$ 10)	7,940
Top-K codes used	50
Label coverage (Top-50 / all)	32.5%
Avg. labels per admission	$\sim$5.1
Avg. note length (words)	$\sim$1,500

4.3 Data Split

We perform a **patient-level** split (no data leakage) using seed 42:

Table 4: Train / validation / test split sizes.

Split	Admissions	Fraction	Label matrix shape
Train	85,081	70%	$(85081, 50)$
Validation	18,371	15%	$(18371, 50)$
Test	18,852	15%	$(18852, 50)$
Total	**122,304**	100%	—

4.4 Preprocessing

Text preprocessing includes: (1) removal of de-identification tokens (`[** ... **]`), (2) lowercasing, (3) stripping non-alphanumeric special characters while preserving medical values (e.g., "WBC-4.8"), and (4) whitespace normalization. For Model B, a *smart truncation* strategy extracts clinically informative sections (discharge diagnoses, hospital course, history of present illness) using regex patterns before tokenization.

[code_freq_tail]

Figure 2: Distribution of the top-50 ICD-10 code frequencies in the training set, illustrating the long-tail class imbalance.

[label_cardinality]

Figure 3: Distribution of label cardinality (number of ICD codes per admission).

[note_length_dist]

Figure 4: Distribution of discharge note lengths (word count) in the training set. The distribution peaks around 1,200–1,500 words, with a long tail extending beyond 4,000 words. The BiLSTM-LAAT's 4,000-token limit covers the vast majority of documents.

5 Results

5.1 Main Results

Table 5 presents test-set performance for all individual models and ensemble variants. All metrics are computed over the full test set (n = 18,852) using the validation-tuned global threshold.

Table 5: Test-set results for all models and ensemble variants (Top-50 ICD-10 codes, n = 18,852). Best values in **bold**.

Model	Thr.	Mi-F1	Ma-F1	Mi-P	Mi-R	Ma-AUP	Mi-AUC
Individual Models							
Model A (TF-IDF+SGD)	0.53	0.60	0.57	0.49	0.75	0.57	0.93
Model B (ClinicalBERT)	0.28	0.52	0.44	0.52	0.52	0.45	0.87
Model C v1 (Chunk+Attn)	0.63	0.53	0.50	0.42	0.72	0.52	0.89
Model C v2 (Focal+Calib)	0.05	0.55	0.43	0.79	0.42	0.56	0.92
Model D (BiLSTM-LAAT)	0.30	**0.72**	**0.66**	0.72	0.72	0.68	0.95
Ensemble Models							
Ens. v1 (A+Cv1)	0.63	0.62	0.59	0.57	0.69	0.59	0.93
Ens. v2 (A+Cv2)	0.43	0.53	0.40	0.81	0.39	0.61	0.94
Ens. v3 (A+Cv1+Cv2)	0.40	0.53	0.40	0.81	0.39	0.61	0.94
Ens. v4 (A+D)	0.35	**0.72**	**0.67**	0.72	0.72	**0.69**	**0.95**
Ens. v5 (A+Cv2+D)	0.35	0.69	0.61	**0.80**	0.62	0.68	**0.95**

Mi = Micro, Ma = Macro, P = Precision, R = Recall, AUP = AUPRC, AUC = AUROC. All values rounded to 2 decimal places.

Key findings:

- **Model D (BiLSTM-LAAT)** is the best individual model with Micro-F1 = 0.72, representing a +0.12 absolute improvement over the TF-IDF baseline and +0.20 over truncated ClinicalBERT.
- **Ensemble v4 (A+D)** achieves the best overall Micro-F1 (0.72) and Macro-F1 (0.67), marginally improving over Model D alone.
- **Model A** shows surprisingly strong recall (0.75) despite its simplicity, confirming that ICD coding relies heavily on specific medical terminology captured by TF-IDF bigrams.
- **Model B** (truncated ClinicalBERT) underperforms all other approaches, confirming that 512-token truncation discards critical diagnostic information.
- **Model C v2** achieves the highest individual precision (0.79) but at the cost of recall (0.42), suggesting the focal loss and calibration shift the operating point toward high-confidence predictions.

5.2 Model Comparison Visualization

[model_comparison_bars]

Figure 5: Bar chart comparing Micro-F1, Macro-F1, and Micro-AUROC across all models and ensemble variants.

5.3 Training Curves

[training_curves_c]

(a) Model C v1 (Chunk-BERT + Label Attention)

[training_curves_d]

(b) Model D (BiLSTM-LAAT)

Figure 6: Training loss and validation F1 curves for the two proposed approaches.

5.4 Confusion Matrices

[confusion_matrix_a]

(a) Model A (TF-IDF + SGD)

[confusion_matrix_d]

(b) Model D (BiLSTM-LAAT)

Figure 7: Per-label confusion matrices (top-10 most frequent codes) with raw counts and row-wise percentages.

5.5 Head vs. Tail Analysis

[head_tail_a] [head_tail_d]

(a) Model A (TF-IDF + SGD) (b) Model D (BiLSTM-LAAT)

Figure 8: Per-label F1 scores stratified by code frequency (head, torso, tail).

5.6 Per-Label Scatter Comparison

[model_d_scatter]

Figure 9: Per-label F1 scatter plots comparing Model D against other models. Points above the diagonal indicate codes where Model D outperforms.

6 Discussion

6.1 Scientific Achievements and Findings

BiLSTM-LAAT outperforms all BERT-based approaches. Despite having only ∼11M parameters (vs. ∼108M for ClinicalBERT), Model D achieves the highest individual Micro-F1 (0.72). This confirms the finding of Vu et al. [7] that RNN-based models with label attention can rival or exceed transformer approaches for ICD coding, while being significantly more efficient (30 min vs. 8 hr training). Two architectural factors explain this advantage: (1) the BiLSTM processes 4,000 tokens in a single continuous pass with no chunking artifacts or cross-chunk boundary issues, whereas Chunk-BERT splits documents into independently encoded windows that lose cross-chunk context; and (2) word-level tokenization preserves clinical terminology directly (e.g., "creatinine" is one token), while BERT's WordPiece subword tokenizer fragments medical terms into multiple subtokens (e.g., "cre", "##atin", "##ine"), requiring the model to reassemble meaning from fragments.

Full-document processing is essential. The truncated ClinicalBERT baseline (Model B,

Micro-F1 = 0.52) underperforms even the non-contextual TF-IDF model (Model A, 0.60), demonstrating that discarding 40–60% of clinical text is catastrophic for ICD coding. Both proposed approaches address this: Model C via chunking and Model D via native 4,000-token processing.

TF-IDF remains a strong baseline. Model A's competitive performance (Micro-F1 = 0.60, Recall = 0.75) highlights that ICD coding depends heavily on specific medical terminology (e.g., "acute kidney failure" → N17.9), which is directly captured by TF-IDF bigrams without contextual modeling.

Calibration matters for clinical deployment. Temperature scaling on Model D reduced ECE from 0.07 to 0.02, producing clinically reliable confidence scores. The gap between AUROC (0.95) and F1 (0.72) indicates strong ranking ability, suggesting that per-label threshold tuning could further improve F1.

6.2 Challenges

- **Label-space scalability:** Full 7,940-label training was computationally infeasible (>100 GPU-hours estimated). We focused on the top-50 codes covering 32.5% of label assignments.
- **Long-document processing:** BERT's 512-token limit loses critical information. The chunking strategy (Model C) introduces boundary artifacts, while the BiLSTM (Model D) handles this natively.
- **Severe class imbalance:** Rare codes appear in <0.1% of admissions. Focal loss ($\gamma = 2.0$) mitigates this but does not fully close the head–tail performance gap.
- **Chunk-BERT training instability:** Model C v1 achieved lower F1 than expected (0.53), likely due to insufficient fine-tuning epochs (only 3 with unfrozen BERT layers). The v2 variant improved marginally with focal loss and calibration.

6.3 Strengths

- Comprehensive comparison of five modeling paradigms (TF-IDF, truncated BERT, chunked BERT, BiLSTM, ensembles).
- Reproducible pipeline with centralized configuration (`src/config.py`), modular codebase, and patient-level data splits.
- Production-ready deployment (Streamlit + FastAPI + Docker).
- Post-hoc calibration for clinical reliability.
- Systematic evaluation including per-label analysis, head/tail stratification, and confusion matrices.

6.4 Shortcomings

- Limited to top-50 codes; clinical deployment requires thousands of codes.
- No cross-attention between chunks in Model C (chunks encoded independently).
- No hierarchical ICD code structure exploitation.
- Ensemble improvements over Model D alone are marginal (+0.002 Micro-F1).
- Per-label threshold tuning not explored (global threshold only).

6.5 Test Set Examples

Example 1 (Correct prediction): A discharge summary containing "acute kidney failure," "essential hypertension," and "type 2 diabetes mellitus" was correctly assigned codes N17.9, I10, and E11.65 by Model D with probabilities 0.94, 0.91, and 0.88 respectively.

Example 2 (Partial miss): A note describing heart failure with mention of "atrial fibrillation" correctly predicted I50.9 (Heart failure, unspecified) and I48.91 (Atrial fibrillation) but missed I25.10 (Atherosclerotic heart disease), which was only mentioned implicitly through "coronary artery disease history."

Example 3 (False positive): Model D predicted E87.1 (Hypo-osmolality/hyponatremia) for a patient with borderline sodium levels (Na=134), where the clinical coder did not assign this code. This illustrates the challenge of threshold sensitivity for borderline findings.

7 Future Work

Ensemble optimization. Our immediate next step is optimizing the weighted ensemble of TF-IDF and BiLSTM-LAAT. These models make complementary errors—TF-IDF captures lexical term matches while BiLSTM-LAAT captures sequential-contextual patterns (e.g., inferring diabetes from "A1c 8.2, continued on metformin" without the word "diabetes" appearing). Preliminary analysis suggests a 3–5% Micro-F1 improvement over the best individual model is achievable through careful weight and threshold tuning.

Per-label threshold tuning. Currently, we apply a single global threshold ($t = 0.30$) across all 50 labels. However, each ICD code has a different base rate—common codes like essential hypertension (I10) may benefit from a lower threshold, while rare codes may need a higher one. Given our strong AUROC of 0.95, per-label threshold optimization could substantially close the gap between ranking quality and binary prediction performance.

Scaling to Top-500 codes. Our experiments use the top-50 codes, covering 32.5% of all label assignments. Scaling to the top-500 codes would provide a more realistic clinical deployment scenario and test how the LAAT attention mechanism handles a 10× larger label space.

Clinical-Longformer. A natural extension is to replace the BiLSTM encoder with Clinical-Longformer, a transformer variant that natively handles 4,096 tokens using sparse attention patterns. This would combine the pretrained contextual understanding of transformers with the full-document coverage that our results show is critical for ICD coding.

Interactive demo with attention visualization. We have built a prototype Streamlit web UI backed by a FastAPI service (Docker-containerized). A clinician can paste a discharge summary and receive predicted ICD-10 codes with confidence scores. Because LAAT provides per-label attention weights, the system highlights which words drove each prediction—providing interpretable evidence critical for clinical trust. Future work includes deploying this demo and conducting user studies with clinical coders.

8 Conclusion

We presented a comprehensive study of ICD-10 code prediction from MIMIC-IV discharge summaries, comparing five individual models and five ensemble configurations. Our key finding is that a lightweight BiLSTM with Label-Aware Attention (LAAT), with only ∼11M parameters, outperforms both a TF-IDF baseline and BERT-based approaches, achieving Micro-F1 = 0.72 with balanced precision and recall. Full-document processing is critical—truncating to BERT's 512-token limit loses 40–60% of clinical text and degrades performance below even a bag-of-words baseline. Label attention is the key architectural component enabling each ICD code to focus on its most relevant tokens. Temperature calibration produces clinically reliable confidence scores (ECE = 0.02). The complete system, including a Streamlit demo, FastAPI service, and Docker deployment, provides a foundation for translating automated ICD coding into clinical practice.

References

[1] Emily Alsentzer, John R Murphy, Willie Boag, Wei-Hung Weng, Di Jin, Tristan Naumann, and Matthew BA McDermott. Publicly available clinical BERT embeddings. In *Proceedings of the 2nd Clinical Natural Language Processing Workshop*, pages 72–78, 2019.

[2] Chao-Wei Huang, Shang-Chi Tsai, and Yun-Nung Chen. PLM-ICD: Automatic ICD coding with pretrained language models. In *Proceedings of the 4th Clinical Natural Language Processing Workshop*, pages 10–20, 2022.

[3] Alistair EW Johnson, Lucas Bulgarelli, Lu Shen, Alvin Gayles, Ayad Shammout, Steven Horng, Tom J Pollard, Sicheng Hao, Benjamin Moody, Brian Bertistle, Leo Anthony Celi, and Roger G Mark. MIMIC-IV, a freely accessible electronic health record dataset. *Scientific Data*, 10(1):1–9, 2023.

[4] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.

[5] James Mullenbach, Sarah Wiegreffe, Jon Duke, Jimeng Sun, and Jacob Eisenstein. Explainable prediction of medical codes from clinical text. In *Proceedings of NAACL-HLT*, pages 1101–1111, 2018.

[6] Adler Perotte, Rimma Pivovarov, Karthik Natarajan, Nicole Weiskopf, Frank Wood, and Noémie Elhadad. Diagnosis code assignment: models and evaluation metrics. *Journal of the American Medical Informatics Association*, 21(2):231–237, 2014.

[7] Thanh Vu, Dat Quoc Nguyen, and Anthony Nguyen. A label attention model for ICD coding from clinical text. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3335–3341, 2020.

A Appendix: User Manual

A.1 Code Structure

```
ICD-CPT-Code-Prediction/
 src/
    config.py          # Centralized hyperparameters and paths
    models.py          # All model architectures (A, B, C, D, Ensemble)
    data.py            # Data loading, preprocessing, dataset classes
 notebooks_local/
    01_data_extraction_local.ipynb
    02_preprocessing_local.ipynb
    03_model_a_tfidf_baseline_local.ipynb
    04_model_b_transformer_local.ipynb
    05_evaluation_demo_local.ipynb
    06_model_c_training.ipynb      # Approach 1 v1
    06_model_c_training_v2.ipynb   # Approach 1 v2
    07_ensemble_evaluation.ipynb
    08_model_d_bilstm_local.ipynb  # Approach 2
 demo.py                # CLI demo (standalone entry point)
 demo/
    streamlit_app.py   # Streamlit web interface
 api/
    main.py            # FastAPI application
    model_service.py   # Model loading and inference service
    schemas.py         # Pydantic request/response schemas
 tests/
    test_data.py       # Text preprocessing tests
    test_models.py     # Model shape and ensemble tests
    test_api.py        # API schema validation tests
 data/models/           # Trained model weights and results
    model_a/           # TF-IDF + SGD artifacts
    model_b/           # ClinicalBERT weights
    model_c/           # Chunk-BERT weights (v1 and v2/)
    model_d/           # BiLSTM-LAAT weights
    ensemble/          # Ensemble config and results
 datasets/processed/    # Preprocessed data artifacts
 Dockerfile             # Docker containerization
 requirements.txt       # Python dependencies
```

A.2 Environment Setup

1. **Clone the repository:**

   ```
   git clone <repository-url>
   cd ICD-CPT-Code-Prediction
   ```

2. **Create a virtual environment:**

   ```
   python3.11 -m venv .venv
   source .venv/bin/activate
   ```

3. **Install dependencies:**

   ```
   pip install -r requirements.txt
   ```

4. **MIMIC-IV data access:** Obtain credentialed access to MIMIC-IV via PhysioNet (https://physionet.org/content/mimiciv/). Place raw CSV files in the path specified in Notebook 01.

A.3 Running the Code

Full pipeline (notebooks in order):

1. `01_data_extraction_local.ipynb` — Extract and join MIMIC-IV tables.
2. `02_preprocessing_local.ipynb` — Clean text, build TF-IDF, create label matrices.
3. `03_model_a_tfidf_baseline_local.ipynb` — Train and evaluate Model A.
4. `04_model_b_transformer_local.ipynb` — Train and evaluate Model B.
5. `06_model_c_training.ipynb` / `v2` — Train Model C (requires GPU).
6. `08_model_d_bilstm_local.ipynb` — Train Model D (requires GPU).
7. `07_ensemble_evaluation.ipynb` — Evaluate all ensemble configurations.
8. `05_evaluation_demo_local.ipynb` — Cross-model comparison and visualization.

Demo (standalone):

```
# CLI demo
python demo.py

# Streamlit web app
streamlit run demo/streamlit_app.py
```

Unit tests:

```
cd ICD-CPT-Code-Prediction
pytest tests/ -v
```

Docker:

```
docker build -t icd-prediction .
docker run -p 8000:8000 icd-prediction
```